



MASTER'S INTERNSHIP REPORT

Development and integration of onboard ROS modules for an autonomous agent participating in the RoboCup@Home

Thomas Ung

August 24, 2022

Master of Computer Science
International Section

Université de Bretagne Occidentale

Supervisor(s)
Teacher(s)

Prof. Cédric BUCHE
François MONIN
Philippe LE PARC

Acknowledgments

I would like to thank Pr. Buche Cédric (CNRS / ENIB, CROSSING) for the supervision, support and opportunity of doing this internship. I would also like to thanks all members of team Robobreizh, Maëlic Neau (Phd student, ENIB/Flinders University), Maël Bouabdelli (Research Engineer, INSA Rouen), Mukesh Barange (Postdoctoral Researcher, INSA Rouen), Louis Li (Machine learning Engineer, IRL-CROSSING), with whom I prepared and participated to the competition. Finally I want to thank researchers and interns at CERV and IRL-CROSSING for supporting the RoboBreizh project and the 2022 RoboCup@Home organizing committee for making the competition.

Contents

Acknowledgments	ii
Contents	iii
Introduction	1
1 The RoboCup@Home Competition	2
1.1 Overview	2
1.2 Qualification	3
1.3 Environment	3
1.4 Schedule & Awards	4
1.5 Tasks & Scoring	5
2 Contribution:	
Solution for the competition	7
2.1 Overview	7
2.2 Architecture	7
2.3 Navigation Node	8
2.4 Dialog Node	15
2.5 Manager Node	17
3 Result and Outcome	20
3.1 Performance	20
3.2 Encountered issues	23
3.3 Outcome and future works	24
Conclusions & Outlook	25
Bibliography	26
A Team Description Paper	27

Introduction

This report is the result of 6 months of work as an intern. Which started on the 21st of February 2022 and ended on the 19th of August 2022. As part of my international master degree, I had to do part of my internship abroad. As a consequence, I spent 3 months in Plouzané, France, at Centre Européen de Réalité Virtuelle (CERV) and 3 months in Adelaide, Australia, at CROSSING an International Research Laboratory (IRL). The goal during the months previous the RoboCup competition was to improve Pepper and develop on board module. My development was mainly focused on navigation and mapping system, create an onboard speech recognition module, as well as integrating different modules implemented by the team. Then, we had to participate to the Robocup@Home competition. Where we all worked on tweaking, improving and debugging our robot for the different tasks during a week. After that, I had to clean and document the work we've done so far.

This document will be structured as follow : will explained the RoboCup@Home competition and described my contribution inside the RoboBreizh team.

By the end of the internship I was offered to continue working on the robocup as a software engineer. Where I would work on a simulation engine and improve the performance of our robot for the competition.

1.1 Overview

The RoboCup competition was founded in 1997, on the same year Deep Blue beat the chess world champion Kasparov as well as the same year NASA landed their first autonomous robot on mars. The competition initially aims to promote and drive science and technology research through soccer. Where the main goal would be for robots to be able to beat humans in a game of soccer. The idea was first mentioned in a paper "On Seeing Robots"[Mac93], and was discussed among a group of researchers at a workshop in japan. This idea rapidly became popular and request came in to organize an international competition. Since, a competition occurs every year in a different country every year. Nowadays, the competition covers multiple diverse topics. The competition is now divided into 5 fields, RoboCupSoccer, RoboCupRescue, RoboCupIndustrial, RoboCup@Home, RoboCupJunior for a total of 19 different leagues.

Created in 2006 RoboCup@Home league aims to develop service and assistive robot technology for future personal domestic applications. The competition usually takes place in an apartment where the robot would need to accomplish Housekeeper tasks (e.g taking garbage out, cleaning the house, etc..) or host task (e.g welcome guests, lead someone somewhere). Today the competition is divided in 3 leagues, Social Standard Platform League (SSPL), Domestic Standard Platform League (DSPL) and Open Platform League (OPL). Standard platform uses a predefined robot, respectively Human Service Robot (HSR) from Toyota Robotics and Pepper from Softbanks. Finally, the open platform is given free choice over components of their robots. While SSPL focus on social interaction, the two other leagues focus more on grasping object and navigation for instance.

This year the competition occurred in Bangkok. This was the first real life competition since 2019. In the next sections of this document, we will focus on the Social Standard Platform League as it is the one RoboBreizh participates in.

During the competition we are challenge in different field : Navigation and

mapping, natural language processing, object recognition, manipulation and motion, speech recognition, gesture recognition.

1.2 Qualification

The competition is scheduled in two phases: the qualifiers and the competition week (also call "finals"). As the rules change every year the qualification process is independent from the actual competition. Traditionally, it started in October with a call of participation from the RoboCup@Home committee. Then, from November to March, teams can send their application materials as follow:

- A team Website
- A team description paper
- A qualification video

The team website should contain a description of the team, the name of the team members and a summary of relevant contributions. The team description paper needs to focus more on the technical approach to solve the main challenges of the competition (as listed above). Finally, the qualification video is supposed to show a robot solving some of the tasks of the competition. In addition, it needs to show the technical abilities described in the paper in a clear way, as a neophyte could understand it.

1.3 Environment

The tasks are done in an arena composed of 4 rooms that you could find in a house, living room, kitchen, bedroom and an office, see [figure 1.2](#). Different furniture and objects are placed across the arena to make the environment closer to what could be a home. Different starting point are used for each task and are marked on an instruction map beforehand [figure 1.3](#), note that this map is not used for navigation. The original and simulated robot are shown in [figure 1.1](#). A detailed description of the robot is shown in.



Figure 1.2: Arena recreation using lidars

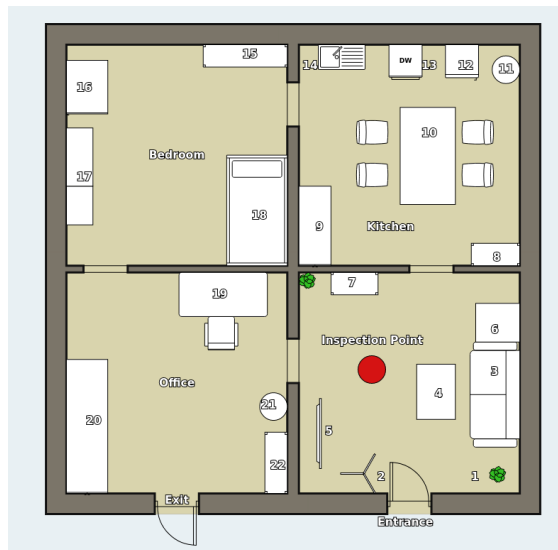


Figure 1.3: Map of the arena (starting point in red).

1.4 Schedule & Awards

The competition is schedule over 6 days. Starts with setting up the robot and testing and adjusting programs for the different task. Then, there is a robot inspection to verify if the robot is safe to operate with (i.e. do not bump into humans, can be



Figure 1.1: Sofbank robot Pepper

stopped at any moment). Once the inspection succeeded, teams can move on to stage 1 that consist of simple tasks split into two categories, housekeeper and party host. Each task is evaluated with different criteria that would give points to the team if meet. Next, the top 6 teams are moving to stage 2 where the tasks are harder to achieve and most of the time are an extended version of the tasks seen in stage 1. Finally, the top 2 teams move to the finals where teams are free to show whatever they want. It is an adequate moment for teams to show features of their robots that couldn't be noticed or wouldn't be useful during the tasks but are interesting to explore.

1.5 Tasks & Scoring

Traditionally, there are around 10 different tasks in the competition (from Stage 1 and Stage 2). Every task has a limited amount of time to do the task. Scores are given by a member of the referee and a member of a team of the @Home league. Given the number of task possible, only the attended tasks will be describe.

1.5.1 Find my mates

For the Find my mates task, people are set in a room sitting or standing and the robot has to go on demand of an operator to the define location, understand what is where in the room, come back and describe person he founds relative to each other.

1.5.2 Receptionist

For Receptionist, the robot has to welcome 2 guest and lead them to the room where the host is, present them to each other and provide a seat to the guest.

1.5.3 GPSR

For General Purpose Service Robot (GPSR), the robot has to follow a automated generated command understand it, execute it and come back to the rally point to get the next command for a total of 3 commands. Each command can be define in 1 to 3 smaller command. In order to address this problem, the robot needs to be very versatile, understand the command properly, being able to retrieve information of the environment, grab object, move towards a location etc. .

1.5.4 EGPSR

Enhanced General Purpose Service Robot (EGPSR) is very similar to GPSR except that the tasks are usually more complex to understand and execute.

1.5.5 Stickler for the rules

During Stickler for the rules, at least 5 party guests are spread around the arena and should to their own will break at least one of these rules:

- All guests have to take off their shoes at the entrance
- All guests must have a drink in hand at all times
- No guest are allowed in the forbidden room
- Guests are not allowed to leave garbage on the floor

If any of the rules is break, the robot has to ask politely to behave appropriately and verify the guest listen to the recommendation.

1.5.6 Where is this

The robot has to explain and show people where they can find places in the Arena (for example, Where is the TV?). The robot has to tell the operator how to get to the desired location and then monitor that the operator reaches the intended location.

2.1 Overview

Starting late February, every members of RoboBreizh started working on solutions in order to address the different tasks of the competition. In order to be efficient, the different critical abilities needed for the robot were developed by different person. While some were working on computer vision and process management, I was the one in charge of the robot navigation. Here I got some help from Maëlic Neau (PhD student, ENIB/Flinders University) for the choice of technologies to use and I used my past knowledge on robot navigation to implement it on board on the robot.

From the analysis of the rules (see [section 1.5](#)), two main abilities were needed: object recognition and object manipulation. For the two tasks of the competition we need to retrieve the labels of objects in the room and pick and place any object. The robot should also be able to navigate without colliding objects on the ground but this ability was not very important as it only provided bonus points.

2.2 Architecture

We decided to use ROS framework to develop our robot applications. The main interest of ROS is its asynchronous communication model, using Subscribers and Publishers we can build a Topic that can process data from acquisition. It is also important to notice that ROS provides synchronous communication in a client/server form: we call it a Service. Finally, an independent and automatic process called Master is ruling those communications via TCP-IP protocol. A schema of this architecture is shown in [figure 2.1](#).

Our architecture will be composed of 5 main packages (called Node in the ROS environment): vision modules (cf [figure 2.2](#)), object manipulation, speech recognition and speech processing module, a navigation and mapping module and finally a manager that schedules process. The manager communicates with the other modules using ros services. While every module would request data

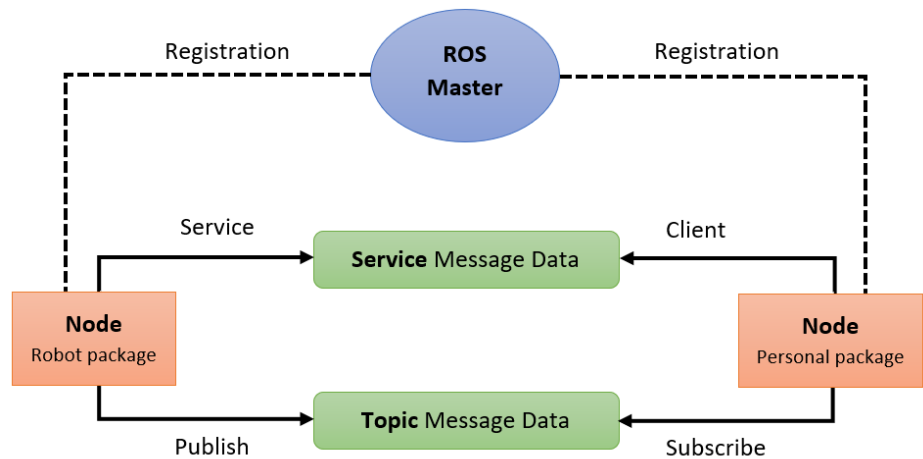


Figure 2.1: ROS communication for Topic and Service.

from the robot sensors using the robot API and wrapping it in ros communication services.

ROS support two different programming languages: Python and C++. In order to achieve program efficiency it is usually better to use the later, this is the one I used for navigation and manager implementation for instance. While c++ can be resource efficient, Python is a very flexible and easy to use language that also allows you to use and train AI models, hence we used it for language processing, object detection and posture detection.

2.3 Navigation Node

2.3.1 Context

Navigation is a critical behavior for the competition as Pepper needs to go to people, lead them to places even simply looking around. In order to achieve navigation, multiple requirements must be set. First and foremost, we need to be able to send velocity commands to the robot. Second, the navigation stack must be provided several type of input, a list of transform matrix to be able to

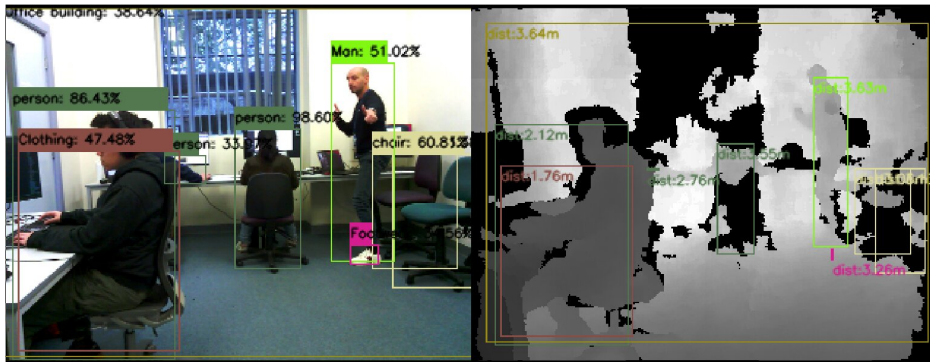


Figure 2.2: Vision module (2D and 3D).

transpose position to different frame, an Odometry¹, a map and various sensors. Following, will be explained what these sources are used for and how to set them.

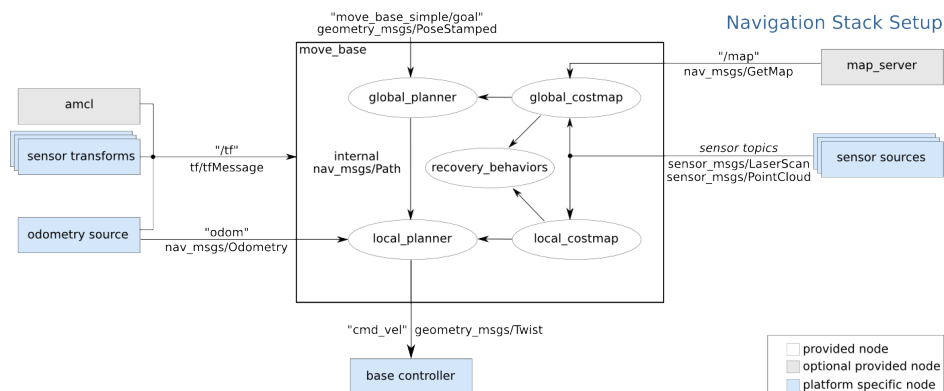


Figure 2.3: Description of the navigation node

- 1 "Odometry is the use of motion sensors to determine the robot's change in position relative to some known position." Source: <https://groups.csail.mit.edu/drl/courses/cs54-2001s/odometry.html#:~:text=Odometry%20is%20the%20use%20of,how%20far%20it%20has%20traveled>.

2.3.2 Octomap

In order to move around, the robot needs some support to know where obstacles, where he is and how to compute path. This is where a map becomes handy, using the odometry of the robot and various sensors such as lidar, sonar, depth camera and rgb camera. Maps are simply binary information stored on a 2D grid [figure 2.4](#) for plane navigation or on a 3D grid [figure 2.5](#), either the pixel/voxel would be free or occupied. To create a 2D map the first thing I did was using

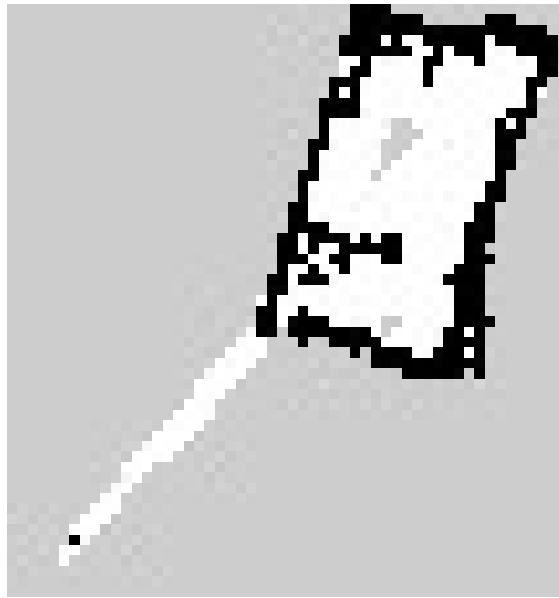


Figure 2.4: 2D Occupancy map

lidars at the bottom of the robot. The main issue with using this method is not being able to map obstacles above a 20cm such as tables and chairs, hence the upper body of pepper would hit the table thinking it exist a free space between table feet. Also, providing a map with binary information is error prone, for instance a reflection on a glass could be seen as an obstacle or obstacles that are detected from afar can be inaccurate by few centimeters. This is why it is preferable to use a probabilistic model in order to estimate whether or not the obstacle would be there.

To address the problems cited above, I decided to use the open source library

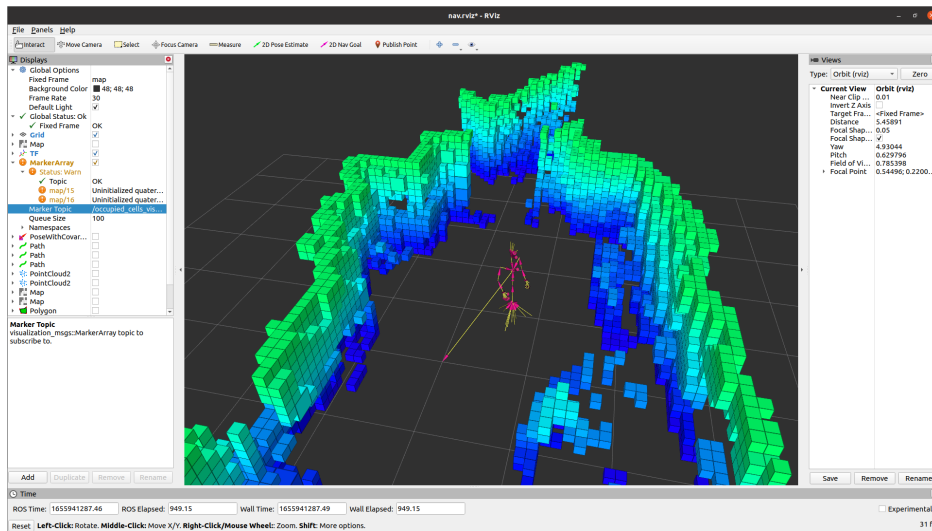


Figure 2.5: 3D Occupancy map

octomap [Hor+13a]. Octomap creates map using octrees with a probability of occupancy, so each voxel can be occupied, free or unknown. Octrees are an efficient way of representing obstacles without consuming too much processing power. One of the issue of representing an obstacle is how detailed you want your obstacle to be, what resolution do you want to have. The higher the resolution is the harder it will be for your your computer compute path planning algorithms. What does octrees [Hor+13b] is that it defines one obstacles as a voxel and divide itself recursively in 8 different voxels until the limit resolution set is reached figure 2.6. If no obstacle is found in one of the voxel then the recursive call is prune. Doing this, you avoid computation cost especially for large object and still have a refine definition for smaller objects.

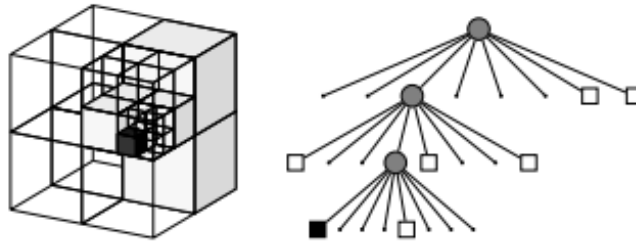


Figure 2.6: Example of an octree storing free (shaded white) and occupied (black) cells. The volumetric model is shown on the left and the corresponding tree representation on the right.

2.3.3 Odometry

There are two kinds of mobile ground robot that use different driving control: Differential drive and holonomic robots (like Pepper). The first one consists of having 2 actuators controlling one wheel each, allowing to move like a car, and the latter allows the robot to move in any direction and rotate on itself. Thanks to the actuators, we can estimate the distance traveled by the robot; this is called Odometry and is usually initialised with the starting position of the robot when he boots up.

Odometry can be error-prone since the relation between motor speed and wheel rotation can be very nonlinear and vary with time. Furthermore, wheels can slip and skid so there may be errors in relating the motion of the wheels to the motion of the robot [BM18].

2.3.4 TF

One other fundamental of robotics is the motion of different joints. Each joint actually creates another referential in space. Which means if you want to set a point from one frame to another you would need a transform matrix. TF is a ROS package that wraps all the transform frames and can provide a dependency tree of them. Using this we are able to define occupancy points defined in the sensor frame to the odom frame (section 2.3.3), and from odom to map.

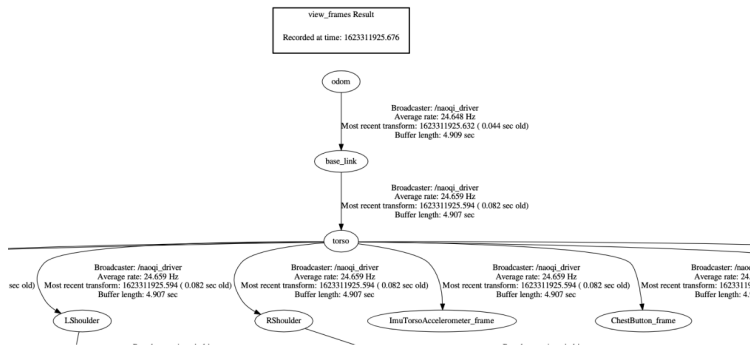


Figure 2.7: TF tree representation without map frame.

2.3.5 Camera Calibration

As mentioned previously, using lidars was not the most appropriate thing to do with Pepper. Thus, I decided to use the point cloud generated by the depth camera instead, because it can provide volumetric information and provides more points than Pepper's lidars.

Noticeable distortion was found during visualization of the point cloud. Giving wrong information on obstacle distance. In order to have proper camera information, we need to adjust intrinsic, extrinsic, and distortion coefficients. For that I used a calibration tool that would take many pictures of a checkerboard in different orientations with various lighting. Given the size of the tiles, an algorithm then computes parameters of a homogeneous transformation matrix that we can change in our program afterwards [17b].

2.3.6 Localization

While performing navigation or mapping it is important to make sure the robot is where it's supposed to be on the map. For that we use Adaptive Monte Carlo Localization (AMCL) package [17a]. It maintains a probability distribution over the set of all possible robot poses, and updates this distribution using data from odometry and lasers.

2.3.7 Mapping

Having all the previous tools set up on the robot, it was time to create a usable 2D map for our robot. In order to achieve this I used :

- octomap to generate a 3D map probabilistic map that is then projected into a 2D map in order to have an accurate representation of the environment
- A teleoperator to manually control the robot while mapping
- AMCL using the current map create as a referral allowing the robot to reallocate properly if necessary

Once the mapping finished, I was able to save the map and correct it. Even with all these corrections with probabilistic models, some errors occurs. This is why it is often needed to manually readjust the map in order to perform better localization when using navigation.

2.3.8 Navigation

Within the previous section we covered all the pre-requirement shown out of the box of [figure 2.3](#) to set the navigation.

The main goal of navigation is to get from the current position to a final position. If the navigation is occurring in an environment where the map never needs to be updated and where you have full knowledge of the environment a path planning algorithm like A* or Dijkstra would very easily be efficient. However in our case the robot is interacting in an environment with humans, meaning humans, or new objects can be introduced in the environment. Hence we need some local path planning in order to avoid these obstacles. In order to be flexible on the navigation we are using two types of costmap.

- A global costmap that takes the information from the static map received as an input and sensors data. This costmap is very similar to the one created but now add inflation around the obstacles to take in account the radius of Pepper.
- A local costmap that is generated only using sensors data. Using

The global costmap gives the possibility to now perform some global planning, here we use dijkstra. This is used to give a rough idea to where the local planner should go and avoid too much computation.

For the local planner I decided to use Dynamic Window Approach (DWA) [17c], this approach computes a set of velocity paths and is reactive to new obstacles.

In order to tackle one of the major issue while interacting within a dynamic environment I decided to use Spatio-Temporal Voxel Layer [MTF20] which would add a new layer of weight of temporary obstacles that would be updated on runtime. Thanks to this layer and dwa, our robot is not able to safely navigate in a known environment.

2.4 Dialog Node

2.4.1 Overview

To start a task Pepper needs to be able to detect when someone is speaking and understand what is said. While the api provide a tool to record the sound and a grammar parsing library, these tools are very limiting in term of maintainability and flexibility. This is why we decided to create a speech detection module and add a natural language processing model in order to understand different commands.

2.4.2 Speech detection

Pepper uses 4 microphones that sends a signal that we will call energy level when detecting any sound. Using these values we can determine changes in sound intensity hence determine whether someone is speaking or not. First we decided to set weight to the different microphones because the user will be facing the robot most of the time, the front microphone has the biggest weight. With the different energy level we determine an average current intensity value that is compared to a threshold. This threshold is reevaluated after each speaking in order to adapt to the ambient noise (cf figure 2.8). The process runs a loop that permanently evaluate the average energy values and transition from state to state : Silence -> possible speech -> speech -> possible silence -> silence. Following is the pseudo code for changing state (cf algorithm 1).

Once the state possible silence is exit, the current buffer is written in order to process be process by Vosk speech recognition model and be converted to text. Vosk [04] is a state of the art offline speech recognition model that supports more than 20 language, support vocabulary reconfiguration and speaker identification.

Algorithm 1: Detect User Speech

```

input :
    counterSpeech, thOffset, counterSilence
    FrontMicImportance, LeftMicImportance
    RightMicImportance, RearMicImportance
    hh
/* Initialisation */
1 STATE = SILENCE
2 firstTime = True
3 rstCounterSpeech = counterSpeech
4 rstCounterSilence = counterSilence
/* Main Loop */
5 while True do
    /* update energy */
6    energy = (getFrontMicEnergy()*FrontMicImportance) + (getLeftMicEnergy()*LeftMicImportance) +
              (getRightMicEnergy()*RightMicImportance) + (getRearMicEnergy()*RearMicImportance) / 4
7    if firstTime then
8        ymin_prev, ymax_prev, ymed_prev = energy
9        firstTime = False
10    end
11    /* update ymed */
12    if energy > ymax_prev then
13        ymax = energy
14    else
15        ymax = hh * ymax_prev + (1 - hh) * ymed_prev
16    end
17    if energy < ymin_prev then
18        ymin = energy
19    else
20        ymin = (1 - hh) * ymin_prev + hh * ymed_prev
21    end
22    ymed = (ymin + ymax) / 2
23    if STATE == SILENCE then
24        if energy > ymed_prev + thOffset then
25            /* update threshold */
26            STATE = POSSIBLE_SPEECH
27            threshold = ymed_prev + thOffset
28            counterSpeech = rstCounterSpeech - 1
29        end
30    end
31    if STATE == POSSIBLE_SPEECH then
32        counterSpeech -= 1
33        if energy > threshold and energy > ymed then
34            if counterSpeech <= 0 then
35                counterSpeech = rstCounterSpeech
36                STATE = SPEECH
37                startRecording()
38            else
39                STATE = POSSIBLE_SPEECH
40            end
41        else
42            STATE = SILENCE
43        end
44    end
45    if STATE == POSSIBLE_SILENCE then
46        counterSilence -= 1
47        if energy > threshold then
48            STATE = SPEECH
49        else if counterSilence == 0 then
50            STATE = SILENCE
51            stopRecording()
52        else
53            STATE = POSSIBLE_SILENCE
54        end
55    end
56    if STATE == SPEECH then
57        if energy < ymed and energy < threshold then
58            STATE = POSSIBLE_SILENCE
59            threshold = ymed
60            counterSilence = rstCounterSilence - 1
61        else
62            STATE = SPEECH
63        end
64    end
65 end

```

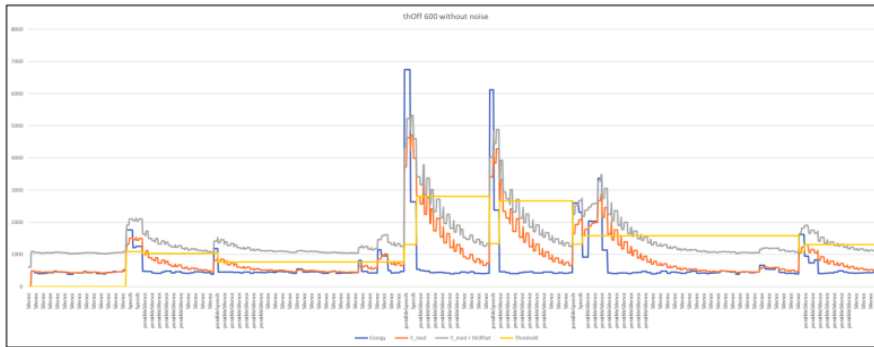


Figure 2.8: Variation of energy level while speaking.

2.4.3 Natural Language Processing

A straight forward but tedious approach to parse commands would be to write the grammar of a define language. Hopefully a NLP model named spacy [HM17] allows us to find relations between words. Once these relations set, we can develop a parsing algorithm that explore a word dependency tree in the sentence and output an intent. The intent generated is a more distinct way a dictionary that gives simpler instructions (e.g. "go to the kitchen" gives "destination":"kitchen"). This tool was mainly useful during the GPSR and EGPSR tasks of the competition.

2.5 Manager Node

2.5.1 Overview

Having different modules working on its own is great but while running an autonomous agent process must be able to be trigger in a certain order. This is why we implemented a finite state machine that we named manager. The manager acts like our main program, given some parameters, it is able to choose whether to trigger navigation to a location, listen to someone, detect object in his environment or manipulate objects. Because we wanted a robust and versatile finite state machine we decided to use a Petri net.

2.5.2 Petri Net Plan

Petri net is a directed graph of place and transitions that allows parallel processing. Unlike other process model, petri net have an exact mathematical definition of their execution semantics. Petri nets execute with markers that moves from place to transition to place. Multiple markers can be requested for a transition acting like a semaphore. The possibility of having multiple marker in the graph makes it nondeterministic meaning it is suitable for concurrent processing.

For each task we defined a plan (cf [figure 2.9](#)) that would describe the different steps of actions that could be encounter as well as sub plans that would be trigger upon certain conditions. Having sub-plans allows us to have a readable and modular petri net. To make link our petri net to ros, we used a package name Petri Net Plan, which allows us to link a set of place and transition to the reference of a c++ function. Each function would then act as an independent process firing one or multiple ros nodes and returning a state to the petri net in order to change state.

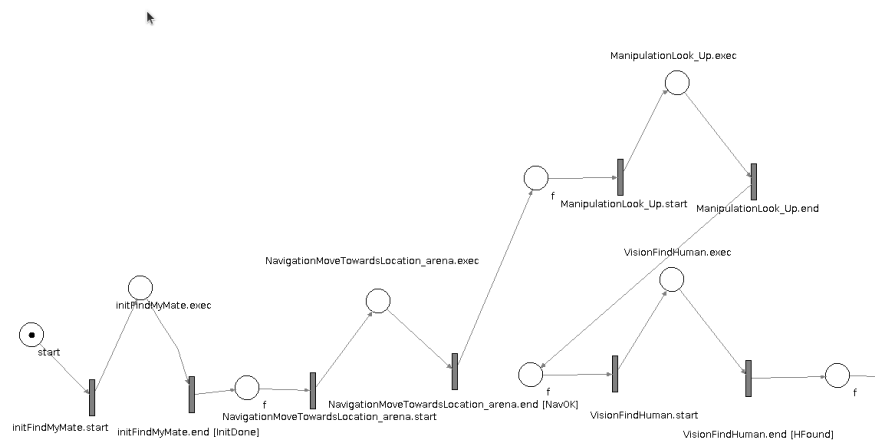


Figure 2.9: Part of the plan for find my mate task

One issue we encountered with using different process is how to manage memory between each of them. To solve this issue we used SQLite an embedded persistent storage solution.

2.5.3 Database

As we wanted everything to be embedded, I choose to use SQLite as a database as it doesn't require any server to run is very lightweight and fast. I had to create a table, corresponding SQL queries as well as the corresponding c++ queries using the SQLite api. The main 3 tables required were person, objects and locations (cf [figure 2.10](#)), these all implement Create, Retrieve, Update and Delete methods. We also have one table for gpsr actions in order to remember the intent from one subplan to another

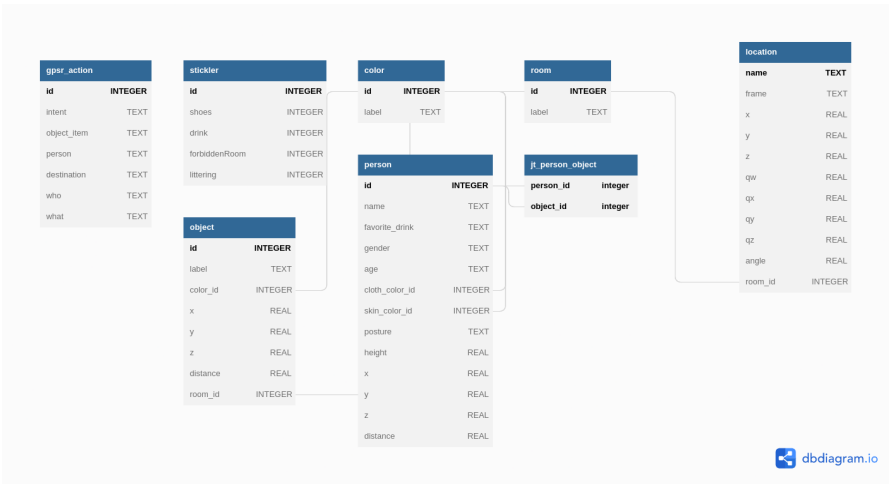


Figure 2.10: Graph of the relation database.

3.1 Performance

Thanks to our solution, RoboBreizh team scored 1st overall at the competition. In the first Stage we scored 1st with a total of 913 points. In Stage 2 we scored 1st with a total of 1363 points to end with 0.775 in finals which gave us the first place in the SSPL league (cf [figure 3.1](#)).

Place	Team	Stage 1	Stage 2	Final
1	RoboBreizh	913	1363	0.775
2	Sinfonia Unia	334	634	0.458
3	LiU@HomeV	87.33		

Figure 3.1: Final competition board (SSPL).

During find my mates, ever our robot performed pretty well with a score of 450. We made him got to the operator, wait for a signal and then move towards the center of the living room. After that, he would simply run multiple perception model to retrieve information from its camera. If someone was detected it would add in the database all information related to that one person and the closest object around them. Then the robot would rotate on itself and look for another person until he found 3 individuals around the room. If any of the person detected already exists in the database then it is not added. Once all visual information are retrieved, Pepper comes back to the operator and describe the first person found in the database and the closest object relative to him. Pepper also performed well during Receptionist task and allowed us scoring 425 points. During this task the Pepper needs to go to the entry (cf [figure 3.2](#)) listen to guest's name and drink (cf [figure 3.3](#)) then lead them to the living room. Once in the living room the robot has to present the guest and the host to each other (cf [figure 3.4](#)). Then offer a seat to the guest (cf [figure 3.5](#)) and come back to the entrance and repeat the process.



Figure 3.2: Receptionist, moving towards the entrance



Figure 3.3: Receptionist, welcome and retrieve informations from the guest



Figure 3.4: Receptionist, present guest to host and host to guest



Figure 3.5: Receptionist, offer a seat to the guest

One issue we faced during this was that our model was not able to understand thai english accent. Which made the task unbelievably complex to start. In the end we decided to bypass informations if they weren't understood and keep

going with the scenario if anything was not understood.

On the other hand GPSR was incredibly difficult and random to perform. For this the host would generate a random command. Say it to the robot then the robot has to parse the intent, and know which plan to trigger. This was a series of plan that we barely tested before so we had very few expectations. However, the robot was able on one command to understand the command properly and trigger all the good events as accordingly but he got lost during navigation, amcl relocalized Pepper at the wrong place, which messed the whole task and didn't give us any points.

Now moving to task two while performing EGPSR the outcome was very similar to GPSR, giving us an idea of how close we actually are from performing this kind of tasks.

Another task that we performed during the second step was Stickler for the rules which gave us 450 points. During the competition we were running out of time for implementing new task. So we choose to make this one as simple as possible, reuse already previous functions thanks to our architecture and the flexibility of our petri net. We decided to only focus on one rule, the forbidden rule, as we could retrieve correct person/object position with our camera and point cloud. Pepper would simply rotate on itself, look if it finds someone, then verify the position. If the person is in the forbidden room, it would kindly ask the person to leave. Wait, verify the person left and ask the person to leave again if he didn't. As for the final, we prepared a script with a series of actions and create a scenario where Pepper would show all his abilities and interact with Cédric, the host presenting what pepper is capable of. During this presentation we showed that all our modules were running on board of the robot, that he was able to perform use kmean to determine an object color, that he is able to detect the posture and the age of the person as well. We showed how our command parsing would work in more detail and explained why having everything on board was an interesting approach and how difficult it was to make it work.

3.2 Encountered issues

During the competition we struggled a lot right on the first day on the navigation. We figured that the mapping and navigation were not robust enough to work on at big environment as in the competition so I had to rework and tweak parameters for the navigation while our team was drawing the map by hand on a

software. The mapping issue is mainly the limitations of Pepper robot that has a hard time performing high cost computation and has too few or bad information coming from his sensors.

Also, as it was our first time attending this competition we were not fully ready for everything. For instance we had to modify and write a lot of codes for every tasks. One important step that we didn't prepare was the inspection task, where the robot had to move towards a point, wait for a signal from the operator and then exit the arena.

3.3 Outcome and future works

After the competition RoboBreizh received request for interview from different media. Different articles already were released ^{6 7 8}.

This competition was a very good environment to learn and apply the knowledge gathered. It was interesting to push ourselves to the limit. The RoboCup was also a good opportunity to meet and share with other teams.

RoboBreizh is going to defend his title in Bordeaux next year, we learned a lot from this competition and already have a lot of features to implement in perspective. For instance, we are going to develop a plugin for unreal engine in order to use it as a simulator. We have improvements to make on NLP, Motion and navigation as well.

6 <https://www.radiofrance.fr/franceinter/un-robot-breton-champion-du-monde-de-la-robocup-1884594>

7 <https://www.ouest-france.fr/high-tech/des-ingenieurs-brestois-champions-du-monde-de-robotique-f23a5e16-19>

8 <https://lejournel.cnrs.fr/articles/robocup-2022-une-equipe-de-brest-championne-du-monde>

Conclusions & Outlook

This internship was overall a great opportunity for me. I was able to learn a lot in an adequate environment. I learned how an autonomous robot architecture, have an brief approach on NLP methods, deepen my understanding of navigation and taught me how to manage a project with few members. I also discovered the group effort under extreme pressure, and thanks to everyone good will of doing well, good communication and sharing, we were able to overcome challenges of the competition.

At the end of my internship I have been offer a job as a research software engineer for IRL-CROSSING lab. Where I would work on the robocup@Home competition in full time. In conclusion, this internship was a good experience as a software engineer and a good introduction to research.

Bibliography

- [04] *Vosk speech recognition toolkit*. 2004. URL: <https://alphacephei.com/vosk/> (see page 15).
- [17a] *amcl is a probabilistic localization system for a robot moving in 2D*. <http://wiki.ros.org/amcl?distro=noetic>. 2017 (see page 13).
- [17b] *Naoqi Extrinsic calibration*. https://github.com/ros-naoqi/extrinsic_calibration. 2017 (see page 13).
- [17c] *ROS documentation dwa*. http://wiki.ros.org/dwa_local_planner?distro=noetic. 2017 (see page 15).
- [BM18] Mordechai Ben-Ari and Francesco Mondada. **Elements of Robotics**. Springer, 2018, 1–308. ISBN: 978-3-319-62533-1 (see page 12).
- [HM17] Matthew Honnibal and Ines Montani. **spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing**. To appear. 2017 (see page 17).
- [Hor+13a] Armin Hornung, Kai M Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. **OctoMap: An efficient probabilistic 3D mapping framework based on octrees**. *Autonomous robots* 34:3 (2013), 189–206 (see page 11).
- [Hor+13b] Armin Hornung, Kai M. Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. **OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees**. *Autonomous Robots* (2013). Software available at <https://octomap.github.io>. DOI: 10.1007/s10514-012-9321-0. URL: <https://octomap.github.io> (see page 11).
- [Mac93] Alan K Mackworth. “On seeing robots.” In: *Computer Vision: Systems, Theory and Applications*. World Scientific, 1993, 1–13 (see page 2).
- [MTF20] Steve Macenski, David Tsai, and Max Feinberg. **Spatio-temporal voxel layer: A view on robot perception for the dynamic world**. *International Journal of Advanced Robotic Systems* 17:2 (2020). DOI: 10.1177/1729881420910530. URL: <https://doi.org/10.1177/1729881420910530> (see page 15).

A

Team Description Paper

RoboBreizh 2022

Team Description Paper

RoboCup@Home SSPL

M. ANDRIES¹ M. BARANGE² M. BOUABDELLI² **C. BUCHE**^{3,*}
A. DIZET¹ D. DUHAUT¹ A. LEGELEUX¹ M. NEAU¹
A. PAUCHET² S. RASENDRASOA² N. WONDIMU¹ T. UNG³

Abstract. RoboBreizh, was founded in 2018. In 2020, we have won the Best Performance award at RoboCup@Home EDU. In 2021, we took the third place at RoboCup@Home Virtual. Currently, we have 10 members from four different laboratories based in France and Australia. This paper aims to introduce the activities that are performed by our team and the technologies that we use. Main contributions include efficient detection, new NLP pipeline and gestures learning by demonstration. Our team is able to work using real robot, qiBullet or Gazebo simulator. RoboBreizh develops his own Pepper Gazebo full environment.

1 Background

RoboBreizh is initially a French RoboCup team of the Brest National Engineering School (ENIB). The team was founded in 2018. Since then, RoboBreizh has won the Best Performance award at RoboCup@Home EDU competitions in 2020. In 2021, RoboBreizh became a joint French team between the ENIB and the National Institute of Applied Sciences (INSA). Due to health condition the competition occurred in simulation and Robobreizh got the 3rd place. In 2022, the team won the 1st prize in SSPL.

¹Lab-STICC, CNRS, France

²LITIS, France

³IRL CROSSING, CNRS, Australia

*Team Leader

2 Research focus and interests

RoboBreizh is aiming at using interactive machine learning, means with the human in the loop, to provide solutions in the context of interactive social robotic. The team offers original and efficient solutions in various contexts. Notably, our robot perception is handled using state-of-art algorithms to detect people/objects and identify genders/ages. Object detection and pose estimation are combined in real time allows to deduce whether a person requires the robot's assistance. Pepper moves its arms to support natural interaction and picks up objects using a Learning by Demonstrations model. Gaussian mixture models (GMM) and Gaussian Mixture Regression (GMR) are used to easily generate movements for manipulation and interaction purposes. This technique enables non-expert users to easily teach movements to the robot by demonstrations. We used existing Learning by Demonstrations algorithms. This programming technique is mainly used with robotic arms which have more degrees of freedom, in this paper we propose an adaptation for the Pepper robot. Regarding the navigation, we used a 3D point cloud converted in a 2d mapping to manage objects where the basement do not represent the potential collision (e.g. chair, tables). The NLP part offers an efficient pipeline combining various complex modules. The team works both on real environment and virtual environment (qiBullet/Gazebo). RoboBreizh developed his own Gazebo framework supporting Pepper's integration.

3 Architecture and environment

3.1 System Architecture

Our architecture is built upon 5 ROS modules: Manager, Perception, Navigation, Movement and Interaction. The manager is a high level structure. Modules output are stored as object instances and available as input information, later. The manager executes tasks in the required order, scheduling Pepper behavior. It also handles task priority. This architecture was designed to easily include new robot behaviors, without editing any external modules. In order to bypass execution lags, orders are cancelled if they take too long to execute. Our architecture uses ROS for its flexibility and the ability to easily segment our code into modules.

3.2 Platform

Our system has been tested using real and simulated environments (cf Figure 1). Concerning simulation two different simulators are used. First, qiBullet [1] is a new simulation environment provided by SoftBank Robotics for Pepper and Nao robot. The advantage of qiBullet is that it can emulate the Pepper system NAOqi. Moreover, this simulator provides a ROS wrapper that makes it possible to run our code. Then, we also use the simulator of ROS, Gazebo [2] (version 7) to emulate the Pepper robot (meshes and sensors) but not the NAOqi system.

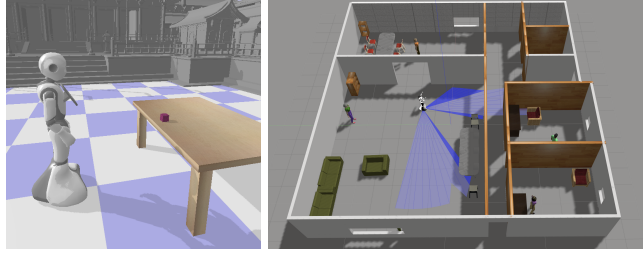


Fig. 1. 3D simulator: (left) QiBullet [1] and (right) Gazebo [2]

4 Approaches

4.1 Perception

Regarding object/person detection, Mask R-CNN [3] was chosen over other state-of-the-art object detection algorithms due to its ability to detect objects with pixel-level precision. Compared to other solutions, Mask R-CNN minimizes the noise added by the background and reduces the risk of inaccurate localisation of object. This level of precision is required when measuring the distance between Pepper and an object. In order to identify item colors we use a kmean model for every identified item and take the biggest cluster value. Our Perception module works in simulation as well as in real-life as shown in Figure 2. To provide information regarding human's activities, we use OpenPose [4], a real time multi-person system which can detect up to 135 different kinds of body keypoints. Our module is also capable of understanding the current state of objects and person. For instance, by using the positions of chairs and persons in an image and how they overlap, it is possible to determine whether the chairs are available. Additionally, OpenPose is exploited to extract the positions of all the hands in an image and thus whether someone is waving. Also, gender and age estimation is performed using models proposed by [5].

4.2 Navigation

3D Mapping As far as navigation with ROS is concerned, the two main approaches are: Visual-SLAM [6, 7] and Laser-based (e.g. LiDAR) SLAM [8]. The first one computes visual information from 2D monocular cameras and 3D Depth sensors to build a 3D map. The second take as input data from LiDAR sensors to build a 2D map. The generated 2D map is quite reliable, however only ground level objects are detected. Objects like tables, chairs or other elevated objects could not be detected. A new generation of map should be able to detect objects at height to avoid collisions. Using Visual-SLAM, Octomap [6] determines a probability of obstacle presence from a point cloud and generates a voxel, visualization in Rviz in Figure 3. First, it is necessary to convert the image of the depth camera into a point cloud and then pass the different arguments in

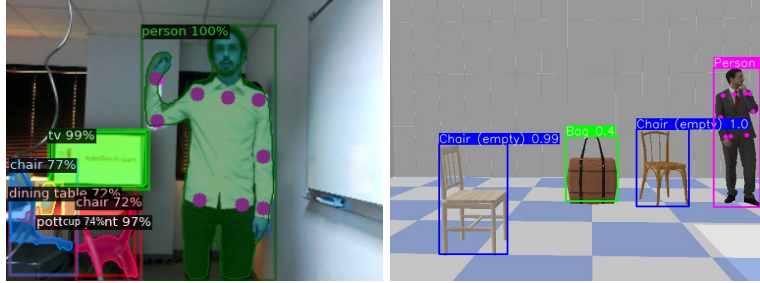


Fig. 2. Human detection and object detection using Mask R-CNN (left) simulation detection (right)

parameter of the octomap_server node. A projection from map 3D to map 2D is done. It is relevant to use octomap's filters (notably height level filter) in order to avoid projecting lamps, beams or even the floor itself on a 2D map. As an alternative to Octomap, Rtab-map [9] is also incorporated in our architecture. This method used points of interests (Figure 3). RoboBreizh is able to use Octomap or Rtabmap, depending on the characteristics of the environment to map.

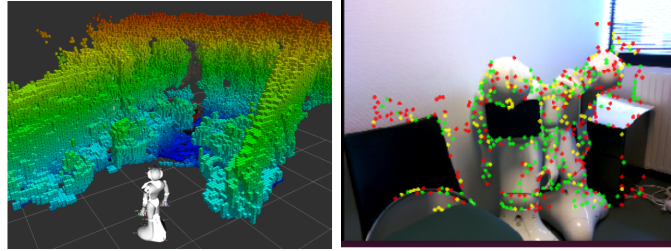


Fig. 3. Octomap: Generation of Voxels (left) and Rtabmap : points of interests (right)

Dynamic Navigation RoboBreizh use a new dynamic local planner [10] inside the ROS Navigation Stack [11] to avoid moving obstacles. This planner use the local costmap and actual data from sensors to predict velocity and trajectory of moving obstacles. Then, the system will generate multiple spatio-temporal trajectories through the global costmap and ranked them to select the best one. At this stage, we choose to fuzz Pepper lasers and depth camera data to feed the Navigation Stack to avoid depth camera's errors [12]. Data from the depth camera are first converted to point cloud and then to laser data using the depthimage_to_laserscan ROS node.

4.3 Movement

Learning by Demonstrations is one of the easiest ways to teach a movement to a robot. We use kinesthetic demonstrations (i.e. human moves the robot's arms). With multiple demonstrations, the robot can generalize the movement. The learning is done at the trajectory level. With this contribution, no code is necessary to define gestures. Gaussian Mixture Model (GMM) in addition to Gaussian Mixture Regression (GMR) learns a movement with multiple demonstrations [13]. GMM generalizes a movement with Gaussians. GMR generates the learning trajectory. [14] modified the GMM/GMR to adapt the movement to environmental changes (object position and obstacle avoidance). Multiple demonstrations, different start and end position, obstacle avoidance impose to choose the modified GMM/GMR. For this scenario, a simple GMM/GMR is implemented. The movement module was developed under some constraints. Pepper can only pick up light objects because of its hands. The Pepper robot has limited movements due to the robot's reachable workspace. The module was developed to simplify movement learning. The learning movement module is composed of two parts: the learning phase and the movement phase. In the learning phase, the user can make multiple demonstrations to the robot for a single movement. The system automatically saves the movement (all different positions) and the user does not need to define way points of the movement. The demonstration is labelled by the user. Each demonstration is processed to remove the beginning and the end when nothing happens to all joints of the robot. The demonstrations are aligned temporally with the first demonstration of the corresponding movement. The user can start the learning process which starts GMM/GMR with previous demonstrations. The initialization of the means is done with K-means algorithm and the selection of the number of Gaussians with the BIC score. The learned movement is saved in a file. The speed of the learned movement can be set. Figure 4 shows the generalization of the movement "Point a seat" of the joint 9 (corresponding to the right shoulder roll) with two demonstrations. The Movement module works in real-time because the learning was done previously. The model works with a single demonstration or with multiple ones.

4.4 Human-Robot Interaction (HRI)

Speech Recognition and NLP

RoboBreizh proposes a new pipeline (cf Figure 5) designed upon Speech Recognition and Natural Language Processing (NLP). This pipeline is connected to Naoqi via a system detecting the user's input voice by analysing the evolution of ambient sound level. In our proposal, the Speech Recognition is handled by the Google API (Google Cloud Speech-To-Text) and NLP by using learning approaches such as Mbot [15] combined with rule-based APIs (e.g. Spacy [16]). In addition to those solutions, a Dialog Act classifier ([17]) is exploited. This classifier enables to adapt the system response to the type of dialog. For example, if the utterance is considered as an Action-Command the system runs Mbot as an

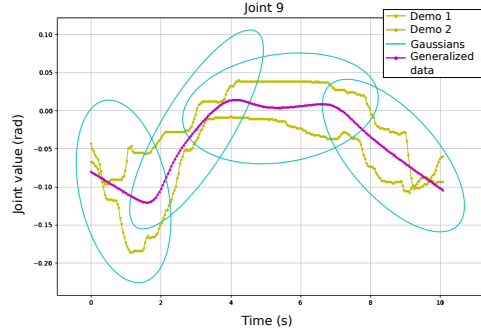


Fig. 4. Learned movement "Point a seat" of the joint 9 with the GMM/GMR. The two demonstrations are in green, the learned movement is in pink. The four Gaussians are displayed in blue.

intent analysis, otherwise the classic rule-based model is used. Using this classifier as a pre-processing unit saves time and prevent intent classifier mistakes. We also implement a Sentiment Analysis Module [18] to classify user's utterance between "negative", "neutral" and "positive" sentiment. This enables to adapt Pepper's response using a rule-based system. An additional component enables to better understand the user intent through the use of commonsense knowledge base and deep-learning based pipeline [19]. Information inside a knowledge base can be represented in a tuple format $e1, r, e2$ where $e1$ and $e2$ are two entities in a relation r . An example of tuple related to the sentence "Going outside" would be: ($E1 = \text{"Go outside"}$, $R = \text{Causes}$, $E2 = \text{"feel better"}$). In this example, the objective would be to generate $E2$ given $E1$ and R . First, we have selected specific relationships from the ConceptNet database. Then, through using the COMET model, we are able to generate common sense facts given the user's command.

Speaker recognition

We adapted classic techniques that works directly on the raw signal data without the need of handcrafted features [20]. The proposed model exploits SincNet, which requires as learning parameters only lower and higher cut frequencies, and therefore reduces the number of parameters learned per each filter and makes this number of parameters independent of the range of each filter. In addition, we combined both the SincNet and a Siamese Network with an algorithm to train Siamese neural networks in speaker identification. The algorithm is funded on the selection of the best anchor for each class. In addition, preparing negative pairs is done based on pairs that are nearest to the anchor class in features space. This selection enhances the performance of the Siamese network since it ensures to learn the confusing cases.

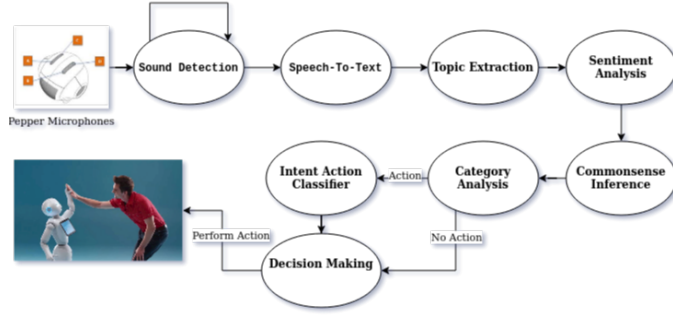


Fig. 5. Diagram of the pipeline

Tablet

It is important for the operator to have feedback on what the robot is doing. To provide feedback we use a web application displayed on the tablet that can receive various information such as what the speaker information, heard sentence or task that is going to be executed.

4.5 Manager

Petri Net

RoboBreizh proposes to use Petri net as a more expressive finite state machine. Allowing parallel processing, rapid scenario creation and flexibility over what to execute at a decide state. Our manager reads a petri net plan and is able to trigger the associated functions to each place. When the manager is running we are able to trigger different petri net plans allowing for flexibility on runtime.

Acknowledgments

This article benefited from the support of the project Prog4Yu ANR-18-CE10-0008, the CNRS and the INCA project of the Normandy region. We also thank the City of Brest (BMO), AFRAN, CERVVAL and Brittany Region.

References

1. Maxime Busy and Maxime Caniot. qibullet, a bullet-based simulator for the pepper and nao robots. *arXiv preprint arXiv:1909.00779*, 2019.
2. N. Koenig and A. Howard. Design and use paradigms for gazebo, an open-source multi-robot simulator. In *International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2149–2154 vol.3, 2004.
3. Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross B. Girshick. Mask R-CNN. *CoRR*, abs/1703.06870, 2017.

4. Zhe Cao, Gines Hidalgo, Tomas Simon, Shih-En Wei, and Yaser Sheikh. Open-pose: Realtime multi-person 2d pose estimation using part affinity fields. *CoRR*, abs/1812.08008, 2018.
5. Gil Levi and Tal Hassner. Age and gender classification using convolutional neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, pages 34–42, 2015.
6. Armin Hornung, Kai M. Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. OctoMap: An efficient probabilistic 3D mapping framework based on octrees. *Autonomous Robots*, 2013.
7. Mathieu Labbé and François Michaud. Rtab-map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation. Labbé and michaud. *Journal of Field Robotics*, 36, 10 2018.
8. Vittorio Perera, Tiago Pereira, Jonathan Connell, and Manuela M. Veloso. Setting up pepper for autonomous navigation and personalized interaction with users. *CoRR*, abs/1704.04797, 2017.
9. Mathieu Labbé and François Michaud. Rtab-map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation. *Journal of Field Robotics*, 36(2):416–446, 2019.
10. Franz Albers, Christoph Rösmann, Frank Hoffmann, and Torsten Bertram. Online trajectory optimization and navigation in dynamic environments in ros. In *Robot Operating System (ROS)*, pages 241–274. Springer, 2019.
11. Rodrigo Longhi Guimarães, André Schneider de Oliveira, João Alberto Fabro, Thiago Becker, and Vinícius Amilgar Brenner. Ros navigation: Concepts and tutorial. In *Robot Operating System (ROS)*, pages 121–160. Springer, 2016.
12. Z. Bauer, F. Escalona, E. Cruz, M. Cazorla, and F. Gomez-Donoso. Refining the fusion of pepper robot and estimated depth maps method for improved 3d perception. *IEEE Access*, 7, 2019.
13. S. Calinon. *Robot Programming by Demonstration: A Probabilistic Approach*. EPFL/CRC Press, 2009.
14. Maria Kyrarini, Muhammad Abdul Haseeb, Danijela Ristić-Durrant, and Axel Gräser. Robot learning of industrial assembly task via human demonstrations. *Autonomous Robots*, 43(1):239–257, 2019.
15. Erick Romero Kramer, Argentina Ortega Sáinz, Alex Mitrevski, and Paul G Plöger. Tell your robot what to do: evaluation of natural language models for robot command processing. In *Robot World Cup*, pages 255–267. Springer, 2019.
16. Matthew Honnibal and Ines Montani. spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing. To appear, 2017.
17. Sujith Ravi and Zornitsa Kozareva. Self-governing neural networks for on-device short text classification. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 887–893, Brussels, Belgium, October–November 2018. Association for Computational Linguistics.
18. C.J. Hutto and Eric Gilbert. Vader: A parsimonious rule-based model for sentiment analysis of social media text. 01 2015.
19. Antoine Bosselut, Hannah Rashkin, Maarten Sap, Chaitanya Malaviya, Asli Çelikyilmaz, and Yejin Choi. Comet: Commonsense transformers for automatic knowledge graph construction. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.
20. Mirco Ravanelli and Yoshua Bengio. Speaker recognition from raw waveform with sincnet. In *Spoken Language Technology Workshop*, pages 1021–1028. IEEE, 2018.

5 Annex

5.1 Team description

The 2022 team consists of the following persons :

Name:	ROBOBREIZH
Leader:	C. BUCHE
Faculties:	A. PAUCHET, D. DUHAUT, M. ANDRIES
Students:	M. NEAU, A. DIZET, A. LEGELEUX N. WONDIMU, S. RASENDRASOA
Post-doct & Engineer:	M. BARANGE, T. UNG, M. BOUABDELLI
Website (RoboCup):	https://www.enib.fr/~robobreizh
Website (@home):	https://www.enib.fr/~robobreizh/src/en/project_home_en.html
Code:	https://github.com/RoboBreizh-RoboCup-Home

5.2 Logo



5.3 Robot's Simulator

- Gazebo v7 (ROS)
- qiBullet (NAOqi + ROS)
- Own Gazebo framework supporting Pepper's integration

5.4 Robot's Description

Hardware: SoftBank Robotics' Pepper (NAOqi 2.5)

Software, External Computer:

- Object/People detection : Mask-RCNN
- People's pose detection : OpenPose
- Movement : GMM / GMR
- Speaker Recognition : SincNet + Siamese (custom)
- Speech Recognition : Google Cloud Speech-To-Text [19] + custom solutions

Software, Onboard:

- People Age/gender estimation : CNN
- Object/People detection : CNN, Kmean
- People's pose detection : OpenPose
- Navigation : ROS Navigation Stack, dwa_local_planner
- Mapping : Octomap
- Speech Recognition : NAOqi, Vosk
- Natural Language Processing : Spacy [\[16\]](#)